

Key design decisions

The system has 34 architectural decisions recorded in `architecture/DECISIONS` with full context, alternatives considered, trade-offs accepted, and revisit conditions. This document highlights the ten most worth defending in a review.

Reference each by ID (D-001, D-005, etc.) — full entries are in `architecture/DECISIONS`.

1. Read-only Postgres role — the load-bearing invariant (D-002)

The decision: create an `agent_readonly` Postgres role with `USAGE` + `SELECT` grants on the business schema, and explicitly `REVOKE` every write privilege (`INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `REFERENCES`, `TRIGGER`, `CREATE`). The agent connects with this role exclusively.

Why it's load-bearing: any two other guardrails can fail and the third layer still catches. The role is not a defence — it is the defence. Application-level guardrails exist to catch destructive intent earlier and give the user useful messages, not because they are the safety mechanism.

Verifiable in 5 seconds:

```
psql -h localhost -U agent_readonly -d shop_db \
-c "INSERT INTO business.customers (name,city,state,join_date) VALUES
('x','x','x','2025-01-01');"
# → ERROR: permission denied for table customers
```

Trade-off accepted: two DB roles to manage (`app` for schema ownership, `agent_readonly` for the agent). Mitigated by keeping both DSNs in one `Settings` class (see D-011).

2. Physical isolation of chat from business data (D-001)

The decision: chat history lives in a separate MongoDB service. The agent has no MongoDB connection string in its environment.

Why architecture, not permission: a second Postgres schema (rejected alternative) still lives behind the same connection surface — one bad grant or migration mistake reopens the exfiltration hole. Different engine, different protocol, no shared surface. A crafted question like *"show me all users' chat history"* cannot exfiltrate other conversations because the target lives outside the agent's reachable universe.

Being an SQL generator, the agent cannot address a document store even if it had one — this is the point. The isolation is enforced by the type of API, not just by credentials.

Trade-off accepted: one extra compose service ($\approx 150\text{MB}$ image, first pull is slow). Justified by the isolation model — the alternative (a second Postgres schema) still lives behind the same connection surface.

3. TypedDict for LangGraph state, not Pydantic (D-005)

The decision: the state schema is a `TypedDict`, not a Pydantic `BaseModel`.

Why: idiomatic in LangGraph — every official example uses `TypedDict`. Zero runtime validation cost. Pydantic re-validates the entire state dict on every node transition; grows linearly with `state["rows"]` size (potentially thousands of dict rows per request) and would show up as measurable CPU cost at production QPS.

Where Pydantic IS used: at the trust boundary — Understand's structured LLM output via `chat_json` uses OpenAI's structured-output mode with a Pydantic schema (see D-016). Validation happens where trust is broken (LLM outputs, user input), not on internal-state transitions where the shape is under our control.

Trade-off accepted: no runtime validation on internal state. Mitigated by strict `-> AgentState` return-type annotations plus editor type-checking (Pylance / pyright).

4. Thin `llm.py` wrapper, not `langchain-core` chat models (D-006)

The decision: one module (`backend/app/llm.py`) with `chat`, `chat_json`, and `chat_stream` helpers. Wraps the raw OpenAI SDK. Nodes call this wrapper, never the SDK directly.

Why:

- **Provider swap is a real goal** — the wrapper hides `LLM_PROVIDER=openai|gemini` branching; nodes stay unchanged when a provider is added.
- **Testable** — the eval harness can mock `llm.chat` once, not every node's `openai.chat.completions.create(...)` call.
- **Langfuse integration lands in one file, not five** — spans, token capture, and error handling live in the wrapper.
- **Avoids pulling in the full LangChain surface** (chains, agents, memory) when only one abstraction is needed.

Trade-off accepted: one more file. Trivial cost.

5. Count-first execution — never silent LIMIT injection (D-019, supersedes D-015)

The decision: Execute wraps every non-aggregation SELECT in `SELECT COUNT(*) FROM (...)` and routes to one of three paths based on the true row count:

- **Small** ($\leq$ `SAFE_INLINE_LIMIT`, default 250): return all rows inline
- **Overflow** ($>$ `SAFE_INLINE_LIMIT`, $\leq$ `HARD_ROW_CEILING=50000`): fetch a sample, set `overflow=true`, Explain uses a variant prompt that states the true count
- **Refuse** ($>$ `HARD_ROW_CEILING`): honest failure with a specific message

Why: auto-appending `LIMIT 500` (the earlier design) would silently give wrong answers on genuinely-larger result sets. A `SELECT DISTINCT customer_id FROM orders WHERE status IN ...` on 5,000 active customers, capped at 500 by silent truncation, would tell the user "500 customers". The user would never know it was wrong.

For a business analytics tool, correctness is not optional. A tool that occasionally lies about counts destroys trust irrecoverably.

Trade-off accepted: one extra Postgres round trip per non-aggregation query ($\approx 10\text{--}50$ ms for the COUNT wrapper). Trivial compared to LLM latency (≈ 1500 ms for Generate). Aggregations skip the wrapper (they're naturally bounded).

6. Bounded self-correction via retry-through-Generate (D-022)

The decision: failed Validate or Execute routes back to Generate through a conditional edge, with the previous SQL + error string injected as a `CORRECTION_TEMPLATE` block in the Generate prompt. Retry cap is `MAX_RETRIES=3`. After the cap, graceful exit to Explain with a canned "I couldn't build a valid query" message.

Why users deserve self-correction: an ambiguous-column error is fixable by the LLM if it sees the error. Wasting a user's time asking them to rephrase is worse UX than a system that retries silently and succeeds.

Why bounded: unbounded retry is unacceptable. Each retry is a full LLM call ($\approx \$0.005$) and ≈ 2 s latency. If the LLM can't fix it in 3 tries with the error in hand, the question is usually structurally unanswerable.

Why it landed in 15 lines: the pattern was pre-shaped by the earlier decision (D-009) to propagate errors via `state["error"]` rather than exceptions. That decision paid off by making the retry loop a graph-edge routing change rather than a rewrite.

Trade-off accepted: extra LLM calls on retry, bounded to $\approx \$0.015$ worst-case per query. Latency capped at ≈ 6 s worst-case.

7. Five-way intent classification (D-018, D-024, D-025)

The decision: Understand returns one of five categories: `query` / `destructive` / `system_access` / `data_unavailable` / `out_of_scope`. Each category has a distinct user-facing refusal message and a per-query `block_reason` sentence.

Why five, not two:

- **Destructive** vs. **system_access** — "delete all customers" and "show me pg_user" are both blocked, but for different reasons. Same generic message would lose the distinction.
- **Data_unavailable** — the hallucination-adjacent case. "Show me customer emails" without this classification would let Generate silently substitute `SELECT name FROM customers` and Explain would confidently answer as if it were correct. The user would never know their real question wasn't answered. **Catching this upstream is the difference between a trustworthy system and one that occasionally lies convincingly.**
- **Out_of_scope** — off-topic questions (weather, jokes) get their own refusal that's honest about the tool's boundaries.

Schema-aware Understand (D-025 refinement): Understand's prompt includes the full `SCHEMA_DDL`. This is what enables `data_unavailable` — the LLM can compare user-requested attributes against what actually exists.

Trade-off accepted: Understand's prompt is longer (≈ 400 extra tokens for `SCHEMA_DDL`). At `gpt-4o-mini` pricing that's $\approx \$0.00006$ per query. Negligible.

8. RAG-assisted generation via pgvector, with retrieval cached in state (D-026, D-034)

The decision: two pgvector tables — `few_shots` (12 curated NL→SQL example pairs) and `column_docs` (15 per-column business notes) — embedded via `text-embedding-3-small`, HNSW indexes with cosine similarity. Populated at backend startup. Retrieved top-k per request and injected into both Generate and Understand prompts. Feature-gated by `RAG_ENABLED`.

Retrieval caching (D-034 refinement): `AgentState` gains a `retrieved_context: dict` field. Understand fetches once and writes to state; Generate reads from state and rehydrates into `RetrievedContext`. Halves the embedding call + pgvector query cost per request. Falls back to fresh retrieval only if state is empty (defensive — matters for direct testing).

Why not decorative: the measured A/B on the "revenue" question shows a \$86,670 accuracy delta between with-RAG and without-RAG. That's not decoration — that's the retrieval layer paying for itself on the first realistic query.

Trade-off accepted: three files to keep in sync on schema change (`init.sql`, `SCHEMA_DDL` in `prompts.py`, `column_docs.json`). Manageable at 4 tables; would collapse to one if we ever adopt schema introspection (see 06_EXTENSION_ROADMAP §3.8).

9. SSE streaming via `asyncio.Queue` + `ContextVar` (D-029)

The decision: the `/query/stream` endpoint returns Server-Sent Events. Backend runs the LangGraph pipeline in a background `asyncio.create_task` and drains an `asyncio.Queue` of lifecycle events. Frontend consumes via `fetch` + `ReadableStream`. For multi-intent, each sub uses a `TaggedQueue` wrapper that auto-injects the `sub` index for per-sub-card routing on the frontend.

Why SSE, not WebSocket: flow is one-way (server → client). WebSocket adds duplex machinery we don't use (frame parsing, close semantics, ping/pong) and breaks through more corporate proxies. SSE fits the pattern exactly.

Why `ContextVar`, not state, for the queue reference: LangGraph's `astream_events` serialises state for observability, and `asyncio.Queue` contains `asyncio.Future` objects that raise `cannot pickle '_asyncio.Future'`. The queue is a communication channel, not pipeline data — it belongs in a `ContextVar`, not `AgentState`.

Why not native `EventSource`: the browser's built-in `EventSource` only supports GET; we need POST for the JSON body. `fetch` + `ReadableStream` is ≈40 lines of manual SSE parsing — no third-party polyfill needed.

Trade-off accepted: two endpoints (`/query` for one-shot, `/query/stream` for SSE). Small; both share ≈90% of the same code path via the `_run_one` / `_run_sub_streaming` helpers.

10. Multi-intent decomposition at the orchestrator layer (D-034)

The decision: compound questions are split by `agent/decompose.py` — a single `chat_json` call with a Pydantic-schema-enforced structured output. Decomposition lives in `main.py`, **not** as a graph node. The 5-node graph stays unchanged; `main.py` calls `graph().ainvoke()` N times in parallel via `asyncio.gather(..., return_exceptions=True)`.

Why orchestrator-level:

- **Keeps the graph clean.** Graph nodes are single-question; the graph itself is a per-request abstraction. Making the graph multi-tenant would require LangGraph's Send API subgraph fan-out — real complexity for no gain when `main.py` can just `gather` N invocations.
- **Per-sub safety is fully preserved.** Every sub goes through Understand → Validate → Execute → Explain. The three-layer defence applies per-sub. Decomposition is a **pre-processor**, not a bypass.

Why parallel via `asyncio.gather`: total latency ≈ `max(sub_latency)` not `sum(sub_latency)`. Real product win on compound questions. Partial success native — one sub failing doesn't sink siblings.

The safety property in one demo: *"Delete all customers and show me revenue"* produces a red destructive block on sub 1 and a normal revenue answer on sub 2 — proves decomposition doesn't weaken the guardrails.

Trade-off accepted: all subs share the same `trace_id` (they appear as parallel generation spans under one Langfuse trace). A stricter parent-child hierarchy would need Langfuse parent-observation-id wiring — deferred as future refinement.

Safety invariants that must be preserved through deployment

These are non-negotiable. Any deployment that breaks one of these is not a valid production configuration.

1. **The `agent_readonly` role must remain read-only.** Any migration that grants writes to this role breaks the entire security model. Verify after every schema migration by attempting an INSERT as `agent_readonly` — Postgres must refuse.
2. **The MongoDB service must remain unaddressable from the agent's execution path.** No connection string in the agent's environment. No injection into the LLM prompt. No proxy access. If chat data is ever moved to Postgres for operational reasons, the isolation model must be re-designed — a second schema alone is not sufficient.
3. **Both guardrail layers must stay live.** sqlglot validation runs in application code; the read-only DB role runs at the infrastructure layer. Disabling either is not a valid production configuration. The two-layer defence-in-depth (plus the intent classifier as a third) is a first-class safety property.

For the full decision record

Every decision in this document has a full entry in `architecture/DECISIONS` with:

- **Context** — what forced the choice
- **Alternatives considered** — what else we looked at, why rejected
- **Trade-offs accepted** — what we lose
- **Revisit if** — trigger conditions to reconsider

The full record has 34 entries covering the six-day build. This document is a synthesis of the ten most narratively important; for depth on any single decision, go to the record.